

MechaFind

Introduction:

Mechanic Finder (MechaFind) is a web-based application designed to help users quickly locate nearby mechanics during vehicle breakdowns or emergencies. It provides a centralized platform where users can find reliable repair services without relying on manual search methods.

Vehicle breakdowns can occur unexpectedly, and finding a mechanic in unfamiliar areas can be difficult and time-consuming. Additionally, many local mechanics lack an online presence, making them hard to discover.

This system solves these problems by connecting users with nearby mechanics in real time using modern web technologies and location-based services.

Scenario Based Case Study:

Imagine a working professional named Rahul who is traveling to a nearby city for an important meeting. While driving through an unfamiliar area, his vehicle suddenly breaks down due to an engine issue. Since he is new to the location, Rahul finds it difficult to identify nearby mechanics, and asking locals takes time and may not guarantee reliable service. This situation creates stress and delays, especially since he has limited time to reach his destination.

At this moment, Rahul opens the **MechaFind application** on his mobile device. After logging in, the system uses location-based services to automatically detect his current position. Within seconds, it displays a list of nearby mechanics along with their profiles, including services offered, ratings, contact details, and availability status. This eliminates the need for manual searching and helps Rahul quickly evaluate multiple options.

Rahul selects a mechanic based on proximity and positive ratings. He sends a service request through the platform by describing his vehicle issue. The selected mechanic receives a real-time notification and reviews the request details. Based on availability, the mechanic accepts the request and shares an estimated arrival time. This seamless communication ensures transparency and builds trust between the user and the service provider.

As the mechanic arrives and resolves the issue, Rahul can track the service status through the application. Once the repair is completed, the request is marked as "Completed" in the system. Rahul is also given the option to provide feedback and rate the mechanic's service, which helps maintain service quality and assists other users in making informed decisions.

Through this scenario, MechaFind demonstrates its ability to provide quick, efficient, and reliable assistance during vehicle breakdowns. By integrating real-time location tracking, digital communication, and service management, the platform significantly improves user convenience and reduces the challenges associated with finding mechanics in emergency situations.

SYSTEM REQUIREMENTS:

Software Requirements:

The development and execution of the MechaFind application require the following software components:

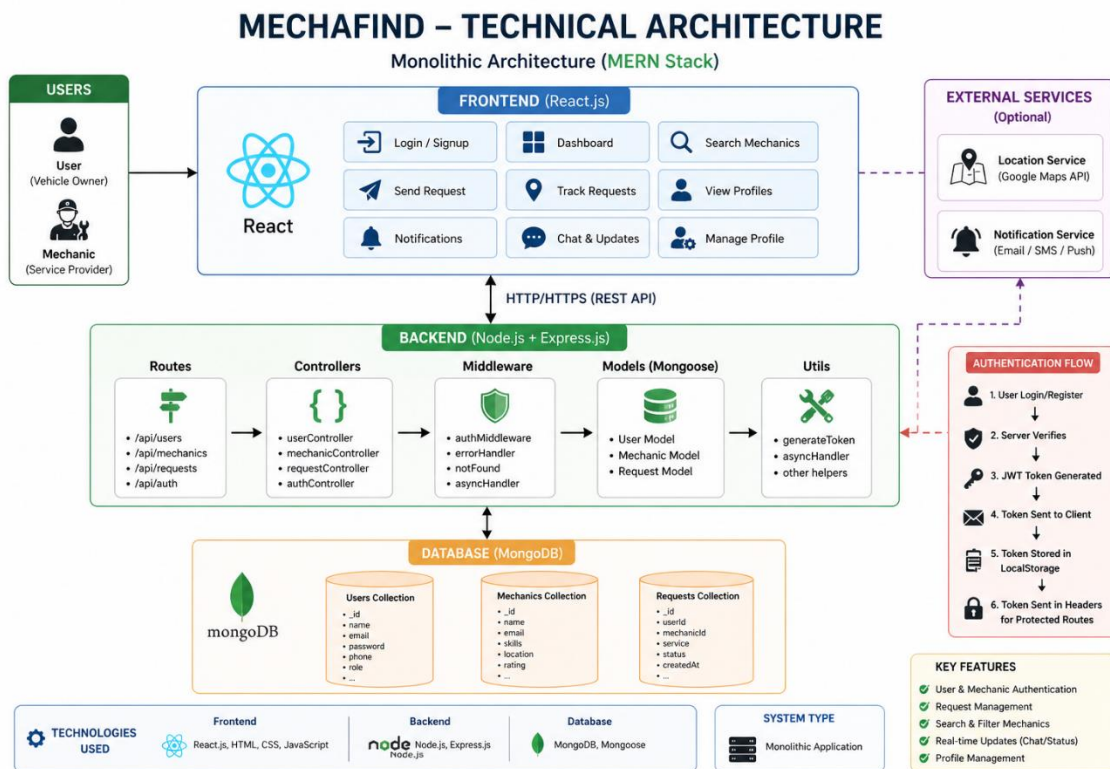
- **Operating System:** Windows 10/11, Linux, or macOS (for cross-platform development)
- **Frontend Technologies:** React.js for building an interactive user interface
- **Backend Technologies:** Node.js with Express.js for handling server-side logic and APIs
- **Database:** MongoDB for storing user, mechanic, and service request data
- **API Testing Tool:** Postman for testing backend APIs
- **Code Editor:** Visual Studio Code (VS Code) for development
- **Package Manager:** npm (Node Package Manager) for managing dependencies
- **Browser:** Google Chrome or Mozilla Firefox (latest version) for testing and execution

Hardware Requirements:

- **Processor:** Intel Core i5 (8th Gen or higher) or equivalent AMD processor
- **RAM:** Minimum 8 GB (16 GB recommended for better performance)
- **Storage:** At least 1 GB of free disk space for project files and dependencies
- **Display:** Minimum resolution of 1366 × 768 for proper UI visualization
- **Internet Connection:** Stable internet connection for API integration and testing

System Architecture:

Technical Architecture:



The MechaFind system is designed using a structured **layered architecture** that ensures clarity, maintainability, and efficient communication between different components. The architecture primarily consists of the **frontend layer**, **backend layer**, and **database layer**, where each layer performs a specific role, enabling a clean separation of concerns.

At the user level, two main actors interact with the system: the **vehicle owner** and the **mechanic**. These users access the application through a web interface developed using **React.js**. The frontend provides features such as user authentication, searching for mechanics, viewing profiles, sending service requests, and tracking request status. It acts as the presentation layer, ensuring a responsive and user-friendly experience.

All user interactions from the frontend are sent directly to the backend through RESTful APIs. The backend is built using **Node.js** and **Express.js**, and it follows a **modular layered structure** consisting of routes, controllers, models, middleware, and utility functions. This structure helps in organizing the application logic efficiently.

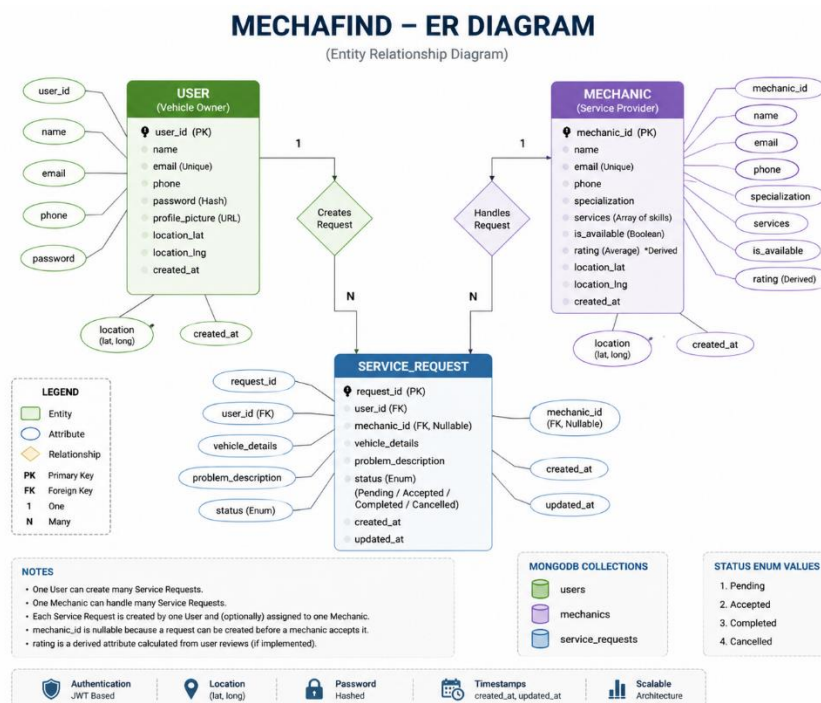
The backend handles all core functionalities, including user authentication using **JSON Web Tokens (JWT)**, profile management, mechanic data handling, and service request processing. Each feature is managed through dedicated controllers and models, ensuring that the code remains modular and easy to maintain. Middleware is used for tasks such as authentication and centralized error handling, improving security and reliability.

All application data is stored in a **MongoDB database**, managed through **Mongoose**. The database contains collections such as users, mechanics, and requests, enabling efficient storage and retrieval of data. This allows the system to handle multiple users and requests effectively.

Overall, the MechaFind system follows a **monolithic yet well-structured architecture**, where a single backend application manages all services. This design ensures simplicity in development while still being scalable for future enhancements, such as integrating external services like location tracking or notifications.

This architecture makes MechaFind a reliable and efficient solution for providing quick and accessible mechanic assistance during vehicle breakdowns.

ER Diagram:



The Entity-Relationship (ER) diagram of the MechaFind system illustrates how data is structured and how different entities interact within the platform. It provides a clear representation of the relationships between users, mechanics, and service requests, ensuring efficient data organization and retrieval.

1. Entities:

The system consists of three main entities:

- **User (Vehicle Owner):**

This entity represents individuals who use the platform to request mechanic services. It includes attributes such as `user_id` (primary key), `name`, `email`, `phone`, `password`, `location` (latitude and longitude), and `created_at`. These attributes store both authentication and profile-related information.

- **Mechanic (Service Provider):**

This entity represents mechanics who provide repair services. It includes attributes such as *mechanic_id* (primary key), *name*, *email*, *phone*, *specialization*, *services* (array of skills), *availability status*, *location* (latitude and longitude), *rating* (derived), and *created_at*. This helps users identify and choose suitable mechanics.

- **Service Request:**

This entity represents requests created by users when they need assistance. It includes attributes such as *request_id* (primary key), *user_id* (foreign key), *mechanic_id* (foreign key, nullable), *vehicle_details*, *problem_description*, *status* (Pending, Accepted, Completed, Cancelled), *created_at*, and *updated_at*. This entity acts as a bridge between users and mechanics.

2. Relationships:

- **User – Service Request (1:N):**

A single user can create multiple service requests, but each service request is associated with only one user.

- **Mechanic – Service Request (1:N):**

A mechanic can handle multiple service requests, but each request is assigned to only one mechanic. Initially, the *mechanic_id* may be null until a mechanic accepts the request.

3. Attributes:

Each entity contains attributes that define its properties. The User entity manages personal and authentication details, the Mechanic entity manages service-related and availability information, and the Service Request entity captures request-specific details such as problem description and status tracking.

4. Primary Keys:

Each entity is uniquely identified using a primary key:

- User → *user_id*
- Mechanic → *mechanic_id*
- Service Request → *request_id*

5. Foreign Keys:

Foreign keys establish relationships between entities:

- *user_id* in Service Request links to the User entity
- *mechanic_id* in Service Request links to the Mechanic entity (optional until assignment)

Key Features:

The MechaFind application provides a set of essential features designed to offer quick assistance during vehicle breakdown situations and ensure smooth interaction between users and mechanics.

1. **Mechanic Search:**

Users can search for available mechanics through the platform. The system displays mechanic profiles, including their skills, services, and availability, helping users choose the most suitable service provider.

2. **Service Request Management:**

Users can raise service requests by describing their vehicle issues. These requests are sent to mechanics, who can view and respond to them. The system supports request creation, status updates, and completion tracking.

3. **Profile Management:**

Both users and mechanics can manage their profiles. Users can update personal information, while mechanics can maintain details such as specialization, services offered, and availability status.

4. **Authentication and Security:**

The application implements secure login and registration using **JSON Web Tokens (JWT)**. This ensures that only authorized users can access protected features and that user data remains secure.

5. **Request Status Tracking:**

Users can track the progress of their service requests through different stages such as *Pending*, *Accepted*, *Completed*, or *Cancelled*, providing transparency and clarity.

6. **Structured Backend Handling:**

The backend system efficiently manages all operations using a modular structure (routes, controllers, models, and middleware), ensuring reliable processing of user requests and data.

7. **Scalable Design (Future Enhancement):**

The system is designed in a way that allows future integration of advanced features such as real-time notifications, GPS-based location services, and feedback systems.

Roles and Responsibilities:

Roles and Responsibilities in MechaFindThe MechaFind system is designed around two primary user roles—vehicle owners and mechanics. Each role has specific responsibilities that ensure the platform operates smoothly and delivers efficient service.

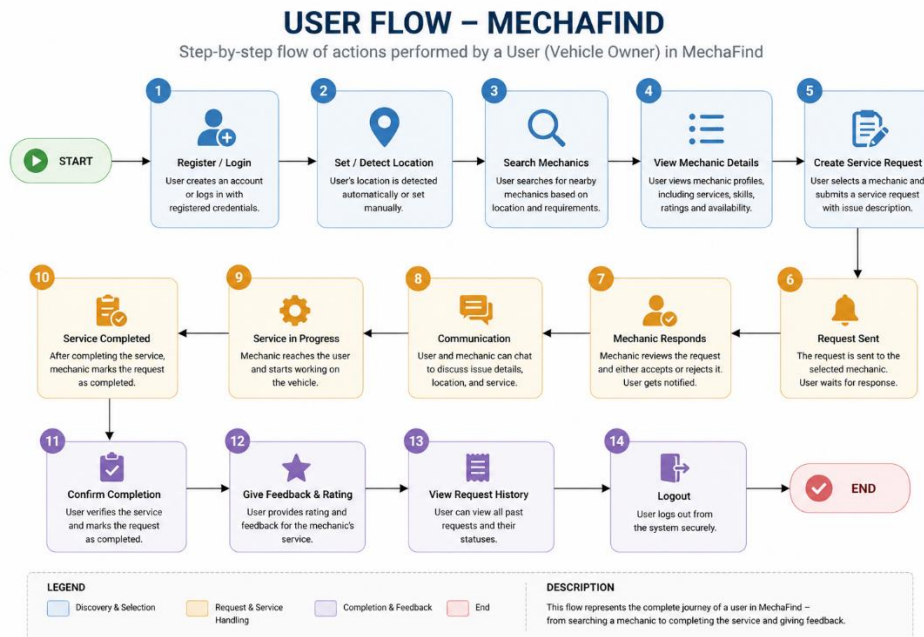
1. User (Vehicle Owner):

- **Account Creation and Maintenance:**
Users register on the platform by providing their personal details. They are responsible for keeping their profile information up to date and ensuring the security of their login credentials.
- **Searching and Requesting Services:**
Users can explore available mechanics and choose one based on their needs. After selecting a mechanic, they create a service request by clearly describing the issue with their vehicle.
- **Providing Clear Information:**
To help mechanics understand the problem accurately, users should provide correct and detailed descriptions of their vehicle issues.
- **Tracking Service Requests:**
Users can monitor the progress of their requests through different stages, such as pending, accepted, or completed, ensuring they stay informed throughout the process.
- **Completion and Feedback:**
Once the service is completed, users can mark the request as finished. They may also share feedback or ratings to help improve service quality.

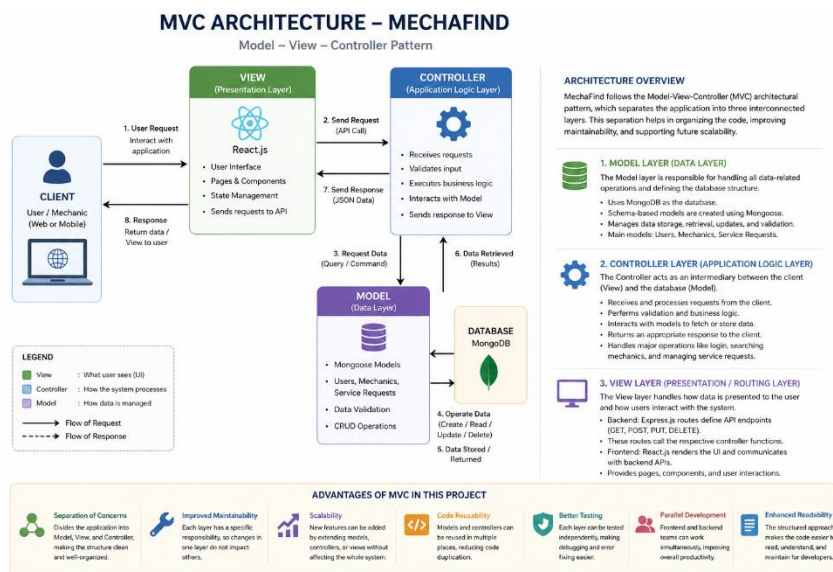
2. Mechanic (Service Provider):

- **Registration and Profile Updates:**
Mechanics register on the platform and maintain their profiles by updating details such as skills, services offered, and availability status.
- **Handling Service Requests:**
Mechanics can view incoming requests from users and decide whether to accept or decline them based on their expertise and current workload.
- **Providing Services:**
After accepting a request, the mechanic is responsible for delivering the required service effectively and professionally.
- **User Interaction:**
Mechanics may communicate with users to better understand the issue and provide updates on the progress of the service.

USER FLOW:



MVC PATTERN:



The MechaFind application is designed using the Model-View-Controller (MVC) architecture, which divides the system into three separate layers for better structure and clarity. This approach helps in organizing the code and improving the overall efficiency of the application. In this system, the Model handles all data-related operations using MongoDB schemas such as users, mechanics, and service requests. The View represents the frontend built with React.js, which allows users to interact with the application through a user-friendly interface. The Controller manages the application logic by handling user requests, interacting with the model, and sending responses back to the view. This separation makes the system easier to manage, extend, and maintain.

Model Layer (Data Layer):

The Model layer is responsible for handling and managing all the data in the application. It defines the structure of the database using MongoDB schemas like User, Mechanic, and Service Request. These models are used to perform operations such as storing new data, retrieving existing records, and updating information. Mongoose is used to create and manage these schemas, ensuring proper validation and consistency of the stored data.

Controller Layer:

The Controller layer acts as the central processing unit of the application. It receives requests from the frontend, processes the input, and applies the necessary business logic. The controller communicates with the models to perform database operations such as fetching or saving data. After processing, it sends an appropriate response back to the client. Tasks like user authentication, searching for mechanics, and handling service requests are all managed within this layer.

View Layer (Routing and Presentation Layer):

The View layer is responsible for how the application is presented to the user. In MechaFind, this is implemented using React.js on the frontend, which provides interactive pages and components. On the backend, Express routes define API endpoints such as GET, POST, PUT, and DELETE. These routes connect user requests to the appropriate controller functions. Together, they ensure smooth communication between the user interface and the backend logic.

Advantages of MVC in this Project:

The MVC architecture provides several benefits that enhance the overall design and performance of the MechaFind application.

- **Clear Separation of Responsibilities:**

Each layer focuses on a specific function, making the application well-structured and easier to manage.

- **Easy Maintenance:**

Since the components are independent, updates or changes in one layer do not affect the others.

- **Scalability:**

The system can be extended by adding new features without disrupting the existing structure.

- **Code Reusability:**

Models and controllers can be reused across different parts of the application, reducing redundancy.

- **Efficient Testing:**

Each layer can be tested separately, making it easier to detect and fix issues.

- **Parallel Development:**

Frontend and backend development can be carried out simultaneously by different team members.

- **Improved Readability:**

The organized structure makes the code easier to understand and work with for developers.

Overall, the use of MVC architecture ensures that the MechaFind application remains clean, efficient, and scalable for future improvements.

Project Setup and Configuration:

Creating project folder

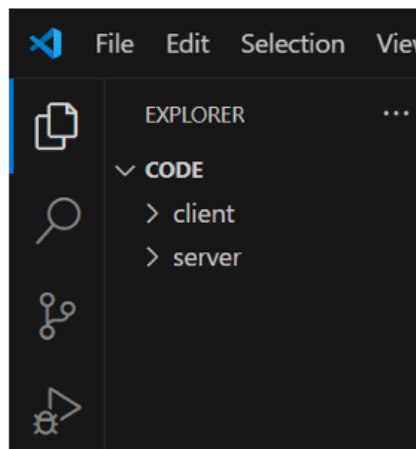
Create a new folder with your <project name>

Inside that folder create two new folders.

Name one as ****Client****.

Name another one as ****Server****.

Now open that folder in VS Code.



Client setup (installing react app)

Open the Client folder in the terminal of VScode.

- `npm create vite@latest . -- --template react`

Select React framework from the given options.

- Select a framework:

- | React

Select JavaScript variant from the given options.

- Select a variant:

■ | JavaScript

Now lets navigate to the client folder by giving the following command

- `cd client`

To install all the packages run the following command.

- `npm install`

To start the React server type the following command.

- `npm run dev`

Server setup (npm init)

Open Server folder in terminal of VScode.

- `npm init -y`

Create files:

- `server.js`

Create folders:

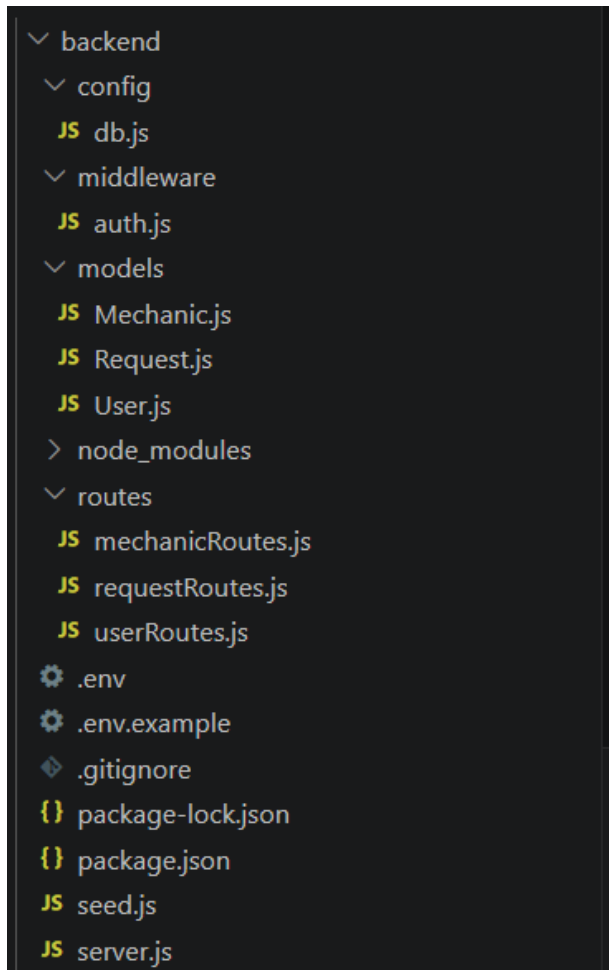
- `Models`
- `Controllers`
- `Routes`

BACKEND DEVELOPMENT:

REFERENCE VIDEO:

https://drive.google.com/file/d/1UhQ7MnUqcQD-Sga8yIVDfcbFDLR5GHZ8/view?usp=drive_link

Backend Structure:



The backend of the MechaFind application is organized into multiple folders, each responsible for handling a specific part of the system. This structured approach improves code readability, maintainability, and scalability.

Config

- **db.js** → Contains the database connection setup using MongoDB and Mongoose. It is responsible for establishing and managing the connection between the application and the database.

Middleware

- **auth.js** → Middleware used for authentication and authorization. It verifies JWT tokens and ensures that only authorized users can access protected routes.

Models

- **User.js** → Defines the schema for user accounts, including details such as name, email, password, and profile information.
- **Mechanic.js** → Defines the schema for mechanics, including their skills, services, availability, and contact details.
- **Request.js** → Defines the schema for service requests created by users, including problem description, assigned mechanic, and request status.

Routes

- **userRoutes.js** → Contains API endpoints for user-related operations such as registration, login, and profile management.
- **mechanicRoutes.js** → Contains API endpoints for mechanic-related actions such as viewing profiles and managing availability.
- **requestRoutes.js** → Contains API endpoints for handling service requests, including creation, updates, and tracking.

Root Files

- **server.js** → The main entry point of the backend application. It initializes the Express server, connects to the database, and configures middleware and routes.
- **seed.js** → Used to insert initial or sample data into the database for testing and development purposes.
- **.env** → Stores environment variables such as database URI and JWT secret for secure configuration.
- **package.json** → Contains project metadata and dependency information.

Overall, this backend structure follows a modular and layered approach, where each component has a clearly defined responsibility. This makes the MechaFind application easier to manage, extend, and maintain.

Database Development:

1. Configure MongoDB:

- Install Mongoose

In your Node.js project directory, run:

```
npm install mongoose
```

Create database connection

Create Schemas & Models

Create Database Connection:

Make sure the database is connected before performing backend operations. The connection setup is as follows:

```
const mongoose = require("mongoose");

const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI, {
      useNewUrlParser: true,
      useUnifiedTopology: true,
    });
    console.log("MongoDB Connected");
  } catch (error) {
    console.error(error);
    process.exit(1);
  }
};

module.exports = connectDB;
```

Create Schemas:

Schemas are defined to structure the data stored in MongoDB. These schemas are designed based on the ER diagram of the MechaFind system.

1) Users.js → Schema/model for user accounts

```
const mongoose = require("mongoose");

const userSchema = new mongoose.Schema({
  name: String,
  email: String,
  password: String,
  phone: String,
  location: {
    lat: Number,
    lng: Number,
  },
  created_at: {
    type: Date,
    default: Date.now,
  },
});

module.exports = mongoose.model("User", userSchema);
```

2) Mechanics.js → Schema/model for mechanic profiles

```
const mongoose = require("mongoose");

const mechanicSchema = new mongoose.Schema({
  name: String,
  email: String,
  phone: String,
  specialization: String,
  services: [String],
  is_available: Boolean,
  location: {
    lat: Number,
    lng: Number,
  },
  rating: Number,
  created_at: {
    type: Date,
    default: Date.now,
  },
});

module.exports = mongoose.model("Mechanic", mechanicSchema);
```

3) Requests.js → Schema/model for service requests

```
const mongoose = require("mongoose");

const requestSchema = new mongoose.Schema({
  user_id: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User",
  },
  mechanic_id: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "Mechanic",
  },
  vehicle_details: String,
  problem_description: String,
  status: {
    type: String,
    enum: ["Pending", "Accepted", "Completed", "Cancelled"],
    default: "Pending",
  },
  created_at: {
    type: Date,
    default: Date.now,
  },
  updated_at: Date,
});

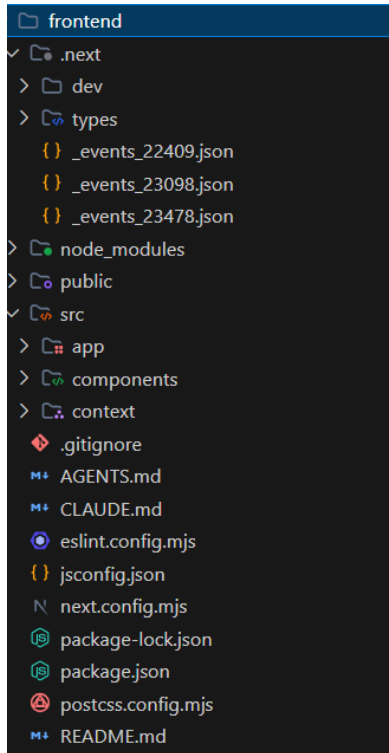
module.exports = mongoose.model("Request", requestSchema);
```

FRONT END DEVELOPMENT:

FRONT END REFERENCE VIDEO:

https://drive.google.com/file/d/14_6D78YDUOY82ht19xSkWe-s6R-lo73B/view?usp=drive_link

Frontend Structure:



Public

- **favicon.svg, icons.svg** → Contains static assets such as icons used across the application.

Source Folder (src/)

This folder contains the main application code, including components, pages, and logic.

Assets

- **hero.png** → Stores images and visual resources used in the application UI.
- **react.svg, vite.svg** → Default assets used for development and design.

Components

- **Map.jsx** → A reusable component used to display maps or location-related data for mechanics and users.

Context

- **AuthContext.jsx** → Manages authentication state across the application. It handles user login, logout, and maintains user session data globally.

Pages

- **Auth.jsx** → Handles user authentication (login and registration).
- **Dashboard.jsx** → Displays main application dashboard after login.
- **Home.jsx** → Landing page of the application.
- **Profile.jsx** → Allows users to view and manage their profile details.
- **Search.jsx** → Enables users to search for mechanics and view available services.

Core Files

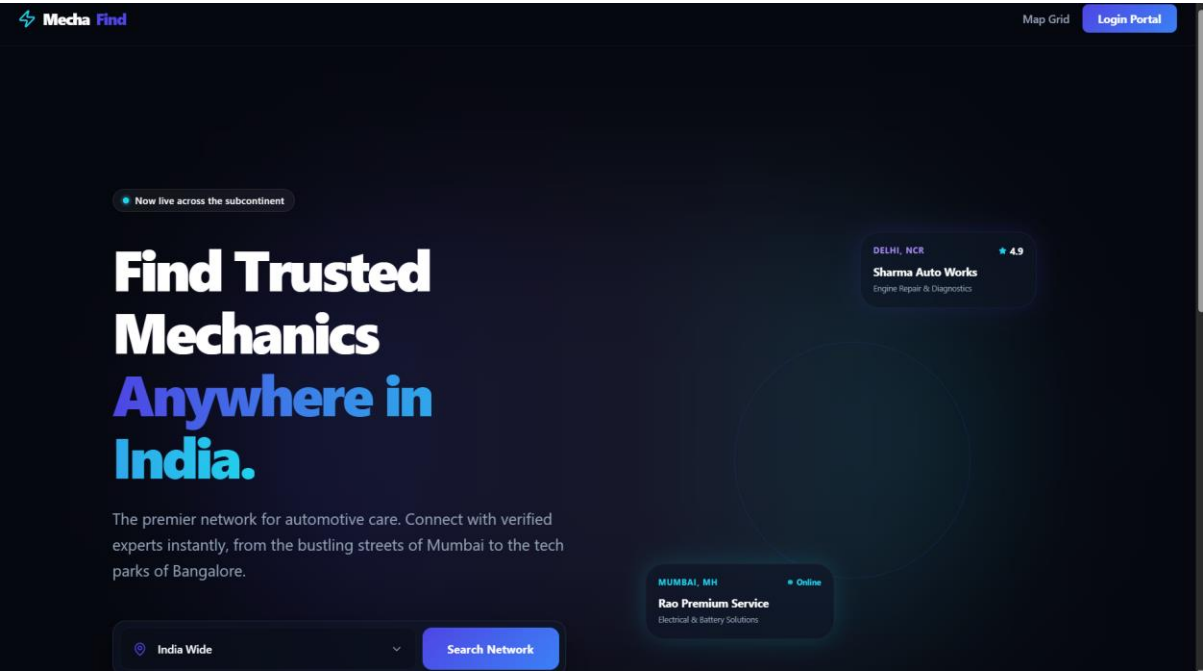
- **App.jsx** → Root component that manages routing and overall layout of the application.
- **main.jsx** → Entry point of the React application, responsible for rendering the App component.
- **App.css, index.css** → Contains global and component-level styling.

Configuration Files

- **vite.config.js** → Configuration file for Vite development environment.
- **eslint.config.js** → Defines linting rules for maintaining code quality.
- **.env** → Stores environment variables such as API base URL.
- **package.json** → Contains project dependencies and scripts.

OUTPUT SCREENSHOTS:

LANDING PAGE:



AUTH.JSX:

User Portal

Mechanic Portal

Join the Grid

Fill in your details to create your user profile.

FULL NAME

John Doe

PHONE NUMBER

CITY/LOCATION

9988776655

Delhi

EMAIL ADDRESS

email@example.com

PASSWORD

.....

Initialize Account

Already registered? Login here

AUTH.JSX:

 **User Portal**

 **Mechanic Portal**

Welcome Back

Enter your credentials to access the MechaFind network.

EMAIL ADDRESS

 email@example.com

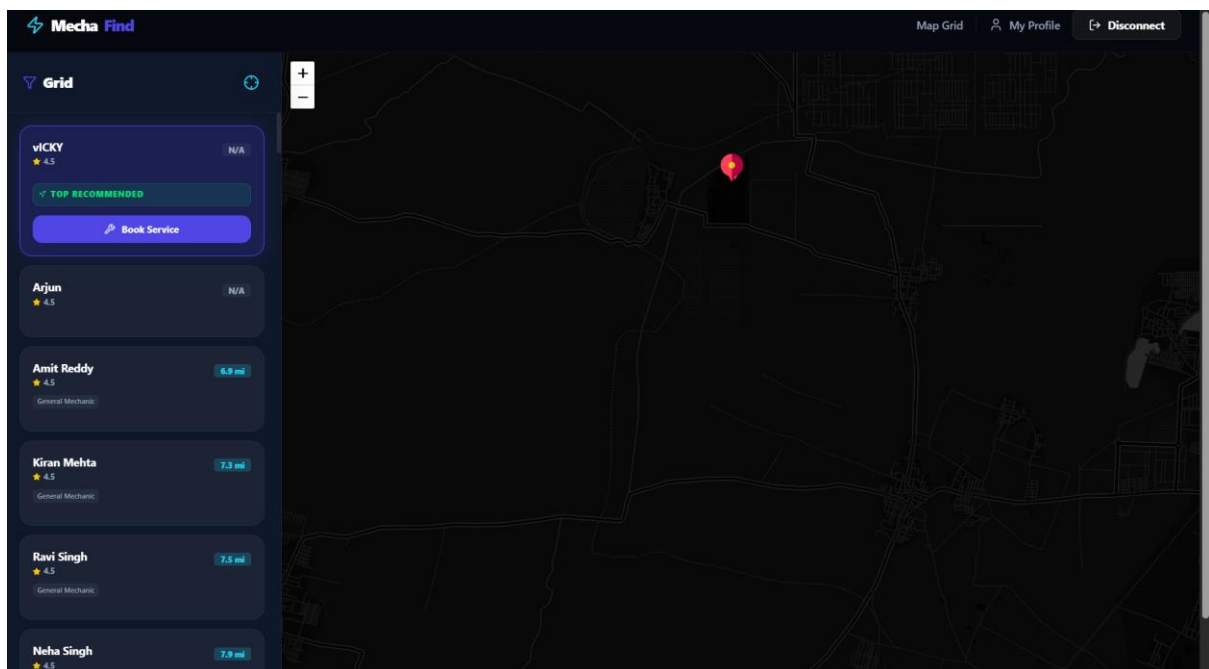
PASSWORD



Authenticate

[New to MechaFind? Register here](#)

SEARCH.JSX:



BOOKING A MECHANIC:

 **Service Dispatch**

Dispatching to **vICKY**. Please describe your issue.

Ex: My car engine is making a weird clicking sound...

Cancel

 **Dispatch**

USER PROFILE PAGE :



Map Grid  My Profile  Disconnect



Anshul Adimulam USER OPERATIVE
 krishnaharshith27m@gmail.com

 Edit Profile

Personal Details

 PHONE
9490667510

 BASE LOC
Vijayawada

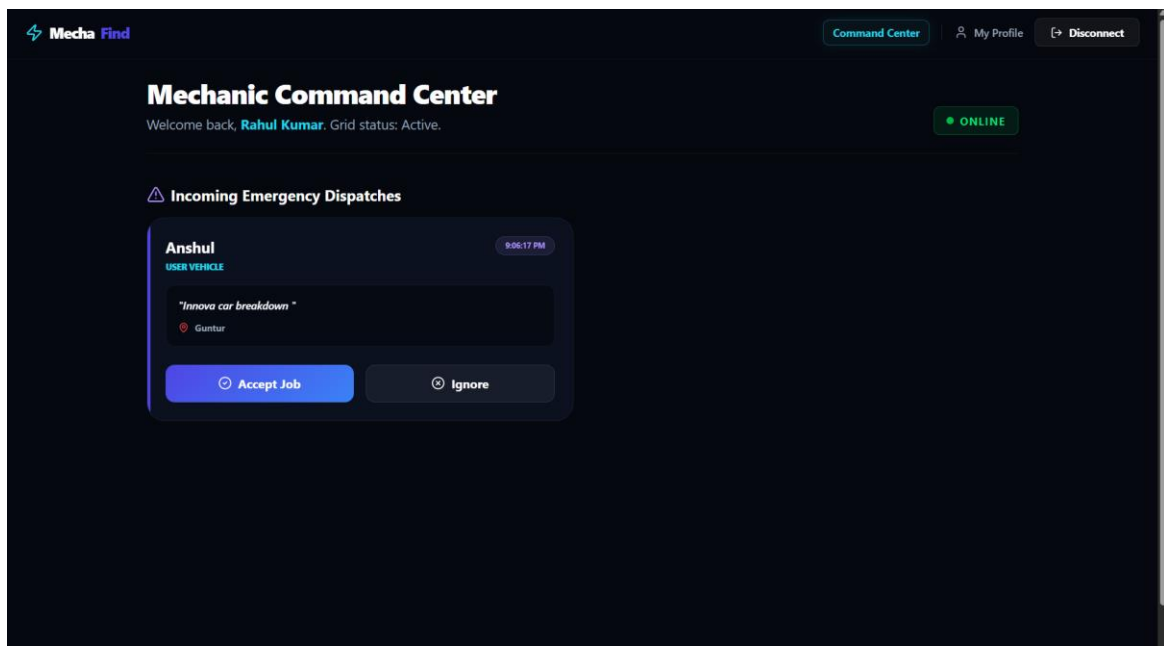
Service History

2 JOBS TOTAL

 **INNOVA CAR BREAKDOWN** PENDING
 4/30/2026  REGISTERED WORKSHOP

 **INNOVA CAR BREAKDOWN** PENDING
 4/30/2026  DEMO ST, VISAKHAPATNAM, AP

MECHANIC DASHBOARD:



DEMO VIDEO LINK:

https://drive.google.com/file/d/1xwENxRz0nqZyvS0TmWfBkKgeGhQXqaY2/view?usp=drive_link

CODE DRIVE LINK:

https://drive.google.com/drive/folders/1uMqMHCM6yVEHbfdjdqqPsohgaIEpQ8rd?usp=drive_link